Informe General de Actividad 8: Programacion Orientada a Objetos en C++ y Simulador Espacial en Consola (class Nave)

Luca Di Gugilemo

Laboratorio de Programación, Escuela de Educacion Secundaria Tecnica Nro. 1 "Eduardo Ader"

Curso: 5to Año 3ra División - Grupos A y B

Profesor: Mansilla Munoz York Elias & Yamil Ganduglia

21 de octubre de 2026

Introduccion

El presente informe academico ha sido elaborado con el proposito de documentar de manera formal y rigurosa el desarrollo integral de la Actividad 8, correspondiente a la asignatura Laboratorio de Programacion (LPR) del 5to ano en la Escuela de Educacion Secundaria Tecnica Nro. 1 "Eduardo Ader", sita en Vicente Lopez. La actividad se inserta dentro de la planificacion curricular del segundo cuatrimestre del ciclo lectivo 2026, constituyendo un punto de inflexion pedagogico en la formacion del futuro Tecnico en Programacion. Su meta primordial reside en guiar al estudiante a traves de la transicion paradigmatica desde la programacion estructurada y secuencial hacia el paradigma de la Programacion Orientada a Objetos (POO), tomando como caso de estudio la construccion de un simulador espacial en consola de comandos de Windows.

Para garantizar una asimilacion significativa y evitar la mera reproduccion mecanica de codigo informatico, la catedra ha estructurado el proceso de aprendizaje bajo la metodologia pedagogica 5R, integrada por las fases consecutivas de Registrar (R1), Reducir (R2), Responder (R3), Reflexionar (R4) y Revisar (R5). Asimismo, el trabajo exige el cumplimiento estricto de las directrices metodologicas de presentacion de informes de la Asociacion Americana de Psicologia (APA, 7ma edicion) y de las normas institucionales plasmadas en la plantilla oficial de la materia. Esta integracion entre desarrollo logico de software y documentacion tecnica normalizada responde a las demandas contemporaneas del mercado tecnologico, donde un sistema informatico carente de documentacion normalizada resulta incompleto en el ambito profesional.

Desarrollo

La sección de desarrollo comprende el análisis exhaustivo de los entornos integrados de desarrollo, la fundamentación de los conceptos teóricos de programación involucrados, la resolución pormenorizada del cuestionario técnico de control y la estructuración del repositorio de control de versiones.

Análisis Comparativo de Entornos de Desarrollo

En la practica profesional de la ingenieria de software, la seleccion del entorno de desarrollo condiciona la productividad, la depuracion y la gestion del ciclo de vida del codigo fuente. Durante la actividad se evaluaron tres herramientas principales: Visual Studio Code, Visual Studio Community y Sublime Text.

Fundamentos del Lenguaje C++ y Proceso de Compilacion

El proyecto implementa conceptos medulares del lenguaje C++ vinculados tanto a la gestion de memoria como al control de flujo en tiempo real. Se utilizaron variables de tipo entero (int) para almacenar coordenadas cartesianas discretas de la consola (x, y) y el estado de salud o vidas de la nave. Las estructuras de control iterativas (bucle while (!gameOver)) gobiernan el motor del juego manteniendo la tasa de refresco a traves de retardos temporales calibrados con la funcion de la API de Windows Sleep(30), la cual evita el consumo excesivo de ciclos de la unidad central de procesamiento (CPU).

Las decisiones logicas de colision y limites espaciales se estructuran mediante sentencias condicionales if encadenadas. Asimismo, la administracion de entidades multiples requirio el empleo de estructuras dinamicas de la biblioteca estandar, especificamente std::vector, permitiendo el dimensionamiento dinamico de punteros a instancias de objetos generadas en el monticulo (heap) mediante el operador new, con su posterior liberacion sistematica a traves de delete y del metodo erase(), previniendo fugas de

memoria fisica.

Resolucion del Cuestionario Tecnico de Control

A continuacion se desarrollan de forma exhaustiva las respuestas correspondientes a las cuatro preguntas tecnicas formuladas en la guia oficial de la actividad:

1. Diferencia tecnica fundamental entre struct y class

En la Actividad 7 se emplearon estructuras definidas mediante struct para empaquetar agrupaciones de datos bajo un identificador comun. En el estandar del lenguaje C++, la unica diferencia sintactica formal entre una estructura (struct) y una clase (class) radica en el modificador de acceso predeterminado: en un struct todos los miembros son publicos (public) por omision, mientras que en una clase todos los miembros son estrictamente privados (private) si no se declara explicitamente lo contrario.

No obstante, en el plano arquitectonico la transicion hacia class encarna un salto cualitativo de paradigma. En la programacion estructurada tradicional los datos y los procedimientos residen disociados; las estructuras actuan como meros contenedores pasivos expuestos a cualquier funcion externa. En contraste, la clase de la Programacion Orientada a Objetos materializa el principio de encapsulamiento: une de manera indisoluble los datos (atributos) con las operaciones autorizadas para manipularlos (metodos), confiriendo a la entidad autonomia de estado y protegiendo su consistencia interna.

2. Encapsulamiento y justificacion de la proteccion de la variable vidas

Declarar la variable vidas o salud como miembro publico quebranta el principio cardinal de encapsulamiento. Si un atributo tan sensible fuera accesible de manera directa e irrestricta desde la funcion externa main(), cualquier programador novato o componente desincronizado podria efectuar asignaciones desprovistas de validacion, tales como jugador.vidas = -10 o jugador.vidas = 9999.

Dicha alteracion descontrolada causaria anomalias operativas en el motor del juego: la

logica de terminacion de partida condicionada a la expresion jugador.getVidas() == 0 quedaria anulada si el valor saltara a numeros negativos (-1 o -5), provocando un bucle infinito en el cual la partida nunca finalizaria e inconsistencias de renderizado en el HUD. Al confinar vidas bajo la etiqueta private e instrumentar el metodo publico reducirVida(), la clase asume el control absoluto de las transiciones de estado, garantizando que el decremento sea estrictamente unitario y controlado.

3. Proposito y ciclo de vida del constructor de clase

El constructor definido en el codigo mediante el encabezado Nave(int _x, int _y, int _vidas) constituye una funcion miembro especial que se ejecuta de forma automatica en el instante exacto en que se instancia un nuevo objeto de la clase Nave en la memoria RAM. Su proposito esencial radica en asegurar que la entidad espacial nazca en un estado valido y predecible, inicializando sus coordenadas espaciales de pantalla y asignandole la cantidad reglamentaria de vidas iniciales.

Sin la existencia del constructor parametrizado, los atributos enteros x, y y vidas albergarian valores residuales arbitrarios provenientes de la memoria no saneada, lo que provocaria que la nave intentara posicionarse fuera del area visible de la consola (por ejemplo, en coordenadas x = 32767) desencadenando fallos de ejecucion de la API de Windows al invocar SetConsoleCursorPosition.

4. Implementacion del desafio de funcionalidad unica (Fase R4)

En cumplimiento de la fase R4 (Reflexionar), se selecciono e integro la funcionalidad denominada Sistema de Escudo Defensivo con Bateria de Energia. Para hacerla operativa, se extendio la definicion de la clase Nave anadiendo dos atributos privados: bool escudoActivo (inicializado en false) y int energiaEscudo (inicializado en 100 unidades).

Asimismo, se agregaron metodos publicos para conmutar el estado mediante la tecla E capturada en el bucle principal a traves de tolower(_getch()). En la subrutina de impacto entre obstaculos y nave, se condiciono la invocacion de reducirVida(): en caso de registrarse

colision con escudo activo, se consumen 25 unidades de energia en lugar de decrementar vidas, neutralizando el impacto y alterando momentaneamente la representacion de la nave a un simbolo de campo de fuerza [^ (X) ^^^^], el cual se desactiva automaticamente al agotarse la reserva de energia.

Estructura del Repositorio y Versionado Dual Git

De conformidad con las exigencias del estandar profesional, la totalidad de los artefactos del proyecto se alojan en la carpeta raiz juegodenaves/, la cual cuenta con configuracion de archivos de control .gitattributes para estandarizar fines de linea LF, .gitignore para excluir binarios ejecutables compilados (*.exe), archivos de objeto y configuraciones de editor, licencia escolar MIT y un archivo README.md que actua como portal de presentacion. El proyecto se encuentra sincronizado simultaneamente en los servicios de alojamiento remoto GitHub y GitLab, permitiendo la verificacion cruzada de los commits y cumpliendo con las pautas del sistema de evaluacion institucional.

Conclusión

La realizacion de la Actividad 8 ha posibilitado la asimilacion practica y profunda de los principios medulares de la Programacion Orientada a Objetos en lenguaje C++, superando la concepcion puramente teorica para volcarla en la resolucion de problemas logicos reales de sincronizacion, renderizado y gestion de memoria. La adopcion de clases, atributos privados y metodos publicos no constituye un mero artificio sintactico, sino una disciplina de diseno indispensable para garantizar la robustez, modularidad y escalabilidad del software. Asimismo, el ejercicio de redactar el presente informe general y los respectivos manuales tecnicos bajo los lineamientos del estilo APA 7ma edicion reafirma que la documentacion es un componente intrinseco del producto de software. La capacidad de comunicar con precision tecnica el funcionamiento interno de un sistema distingue al programador profesional, consolidando las competencias profesionales que la Escuela de Educacion Secundaria Tecnica Nro. 1 promueve en la especialidad de Programacion.

Referencias

Luca Di Guglielmo

Github: https://github.com/specIux/actividad-8-lpr

Gitlab: https://gitlab.com/specIux/actividad-8-lpr#

Laboratorio de Programación 5°3°

ACTIVIDAD 8: PROGRAMACIÓN ORIENTADA A OBJETOS (POO) EN …